Estructuras de Datos
PECL

SIMULACIÓN DE TRIAJE Y ATENCIÓN A PACIENTES EN LOS QUIRÓFANOS DE UN HOSPITAL

Nicolás Ibáñez Zurdo [71724921L]
Alex David Castro González [06712502K]

PARTE 1

Detalles y justificación de la implementación

Especificación concreta de la interfaz de los TAD's implementados

TAD's creados

Paciente
Representa a cada paciente del hospital almacenando su DNI con letra, un identificador único (ID), un número de habitación único y un booleano que indica si sufre apendicitis o hernias. Todos sus atributos se asignan de forma automática.

Pila y NodoPila
Estructura de datos dinámica de carácter LIFO en la que se almacenan inicialmente los pacientes generados de manera aleatoria antes de ser derivados a las salas de preoperatorio.

Cola y NodoCola
Estructura de datos dinámica de carácter FIFO utilizada para modelar las cuatro salas de preoperatorio del hospital: Salas A y B para las pacientes con apendicitis, y Salas C y D para las pacientes con hernias.

Lista y NodoLista
Estructura dinámica simplemente enlazada que mantiene a los pacientes ordenados por prioridad (según su ID) para simular el orden de entrada a los dos quirófanos disponibles. Siendo estos el quirófano 1 que es la lista de apendicitis y el quirófano 2 que es la lista de hernias.

Controlador
Es donde se lleva a cabo la instancia de la pila, las cuatro colas y las dos listas, gestionando la lógica y la comunicación con el menú principal.

Definición de las operaciones de los TAD's creados

Operaciones del TAD Paciente

- Paciente(): Constructor sin argumentos el cual genera automáticamente el DNI, inicializando ID y habitación a cero.
- ~Paciente(): Destructor de la clase, sin argumentos ni retorno.
- bool tieneApendicitis(): No recibe argumentos y devuelve true si el paciente tiene apendicitis o false si tiene hernias.
- int getId(): No recibe argumentos y devuelve el identificador entero de prioridad del paciente.
- int getHabitacion(): No recibe argumentos y devuelve el número entero de habitación asignada.
- void setId(int nuevoId): Recibe el entero nuevoId asignado en el preoperatorio y no devuelve nada.
- void setHabitacion(int nuevaHab): Recibe el entero nuevaHab asignado en el preoperatorio y no devuelve nada.
- void mostrar(): No recibe argumentos e imprime por consola los datos del paciente según su estado actual.
- void generarDNI(): Método privado sin argumentos ni retorno que calcula un número aleatorio de 8 dígitos y su letra oficial.

Operaciones del TAD Pila

- Pila(): Constructor que inicializa el puntero último a NULL y la longitud a cero.
- ~Pila(): Destructor que elimina todo su contenido y finalmente la pila, liberando así su memoria.
- void insertar(Paciente v): Recibe un objeto Paciente, lo apila en la cima y aumenta la longitud. No devuelve nada.
- Paciente extraer(): Sin argumentos, desapila el nodo en la cima, libera su memoria y devuelve el Paciente almacenado.
- Paciente cima(): Sin argumentos, devuelve el objeto Paciente situado en la cima sin eliminarlo de la pila.
- void mostrar(): Sin argumentos ni retorno, recorre la pila desde la cima mostrando los datos de cada paciente.
- int getLongitud(): Sin argumentos, devuelve un entero con el número actual de pacientes en la pila.
- void vaciar(): Sin argumentos ni retorno, su función es extraer y eliminar todos los nodos existentes en la pila.

Operaciones del TAD Cola

- Cola(): Constructor que inicializa los punteros primero y último a NULL y la longitud a cero.
- ~Cola(): Destructor que elimina todo su contenido y finalmente la cola, liberando así su memoria.
- void insertar(Paciente v): Recibe un Paciente y lo coloca al final de la cola, no tiene retorno.
- Paciente eliminar(): Sin argumentos; extrae el primer nodo de la estructura y devuelve su Paciente.
- Paciente verPrimero(): Sin argumentos, devuelve el Paciente en la cabecera de la cola sin extraerlo.
- void mostrar(): Sin argumentos ni retorno, recorre la cola de principio a fin mostrando cada paciente.
- int getLongitud(): Sin argumentos, retorna el número entero de elementos almacenados en la cola.
- void vaciar(): Sin argumentos ni retorno, desencola todos los elementos hasta dejar la cola vacía.

Operaciones del TAD Lista

- Lista(): Constructor que inicializa el puntero primero a NULL y longitud a cero.
- ~Lista(): Destructor que elimina todo su contenido y finalmente la lista, liberando así su memoria.
- void insertarOrden(Paciente v): Recibe un Paciente y lo inserta ordenado por ID de mayor a menor. No tiene retorno.
- void mostrar(): Sin argumentos ni retorno. Muestra por pantalla todos los pacientes de la lista en orden.
- Paciente getMayorPrioridad(): Sin argumentos. Devuelve el primer Paciente de la lista (mayor ID).
- Paciente getMenorPrioridad(): Sin argumentos. Recorre hasta el último nodo y devuelve el Paciente de menor ID.
- int getLongitud(): Sin argumentos. Devuelve la cantidad de pacientes en la lista.
- bool estaVacia(): Sin argumentos. Devuelve true si la lista está vacía y false en caso contrario.
- void vaciar(): Sin argumentos ni retorno. Libera todos los nodos y restablece la longitud a cero.

Operaciones de la clase Controlador

- int pacientesEnPila(), pacientesEnSalaA(), pacientesEnSalaB(), pacientesEnSalaC(), pacientesEnSalaD(), pacientesEnListaApendicitis(), pacientesEnListaHernias(): Métodos consultores sin argumentos que devuelven el número entero de pacientes en cada estructura.
- void genera12Pacientes(): Sin argumentos ni retorno, crea 12 pacientes aleatorios y los apila hasta un tope de 48.
- void muestraPacientes(): Sin argumentos ni retorno, invoca el método de visualización de la pila.
- void borraPacientesPila(): Sin argumentos ni retorno, vacía la pila y reinicia el contador de pacientes generados.
- void encolarPacientes(): Sin argumentos ni retorno, desapila pacientes, asigna ID y habitación, y los reparte en las colas.
- void muestraPacientesSalasAyB() y void muestraPacientesSalasCyD(): Sin argumentos ni retorno, imprimen el contenido de las salas de apendicitis y hernias respectivamente.
- void borraPacientesColas(): Sin argumentos ni retorno, vacía las cuatro colas de las salas de espera.
- void enlistarPacientes(): Sin argumentos ni retorno, extrae los pacientes de las cuatro salas y los inserta ordenadamente en su lista de quirófano.
- void muestraPacientesApendicitis() y void muestraPacientesHernias(): Sin argumentos ni retorno, muestran los pacientes de cada quirófano.
- void buscarPacientes(): Sin argumentos ni retorno, busca e imprime el paciente de apendicitis de menor prioridad y el de hernias de mayor prioridad.
- void reiniciar(): Sin argumentos ni retorno, vacía todas las estructuras y restablece los registros de IDs y habitaciones a su estado inicial.

Solución adoptada: descripción de las dificultades encontradas

Garantía de unicidad en IDs y habitaciones y límite de generación

El enunciado exige que tanto el ID de prioridad (1 a 49 para apendicitis y 51 a 99 para hernias) como el número de habitación (101 a 199 en primera planta para apendicitis y 201 a 299 en segunda planta para hernias) sean únicos para cada paciente, además de limitar la generación en la pila a un máximo de 48 pacientes. Para evitar coincidencias con rand(), se implementaron dos vectores booleanos de control (idsUsados[100] y habitacionesUsadas[300]) dentro de Controlador. Al extraer un paciente hacia el preoperatorio, se genera un número aleatorio dentro de su rango y se comprueba si ya ha sido asignado, repitiendo la extracción solo en caso de colisión.

Criterio de ordenación por prioridad en las listas de quirófano

Al analizar el ejemplo de simulación del guión (Opciones I, J y K), observamos que las listas de quirófano muestran a los pacientes ordenados de forma descendente según su ID (de 44 a 1 en apendicitis y de 99 a 52 en hernias). Asimismo, en la opción K el paciente con apendicitis de menor prioridad corresponde al ID 1 y el de hernias de mayor prioridad corresponde al ID 99. En consecuencia, se diseñó el método insertarOrden de la clase Lista para insertar en orden decreciente de ID, situando los valores más altos al principio de la lista simplemente enlazada.

Explicación de los métodos más destacados

Asignación de preoperatorio y reparto equilibrado (Controlador::encolarPacientes**)**
Este método ejecuta un bucle mientras la pila no esté vacía. En cada iteración extrae el paciente de la cima mediante pila.extraer(), consulta su dolencia con tieneApendicitis() y le asigna su ID y número de habitación exclusivos mediante generarIdUnico() y generarHabitacionUnica(). Luego compara la longitud de las dos salas candidatas (salaA.getLongitud() <= salaB.getLongitud() para apendicitis o salaC.getLongitud() <= salaD.getLongitud() para hernias). El operador <= garantiza que el paciente ingrese siempre en la cola con menos enfermos y que, en caso de igualdad, entre preferentemente en la Sala A o en la Sala C para hernias tal como especifica el enunciado.

Inserción ordenada en quirófano (Lista::insertarOrden)
Recibe un objeto Paciente y crea dinámicamente un nuevo NodoLista. Con dos casos:

1. **Inserta al principio:** Si la lista está vacía (!primero) o el ID del nuevo paciente es estrictamente mayor que el del primer nodo (v.getId() > primero->valor.getId()), el nuevo nodo apunta a primero y pasa a ser la nueva cabecera de la lista.

2. **Inserta al final:** En caso contrario, un puntero auxiliar aux recorre la lista mientras exista un nodo siguiente cuyo ID sea mayor que el del paciente a insertar (aux->siguiente && aux->siguiente->valor.getId() > v.getId()). Al detenerse, enlaza el nuevo nodo justo después de aux, preservando el orden descendente en toda la estructura.

Búsqueda por prioridades extremas (Controlador::buscarPacientes)
Al mantenerse las listas ordenadas de mayor a menor prioridad (ID decreciente), la búsqueda de la Opción K resulta directa, de forma que para obtener el paciente con hernias de mayor prioridad, se invoca listaHernias.getMayorPrioridad(), que devuelve directamente el valor del nodo cabecera (primero->valor) en complejidad $O(1)$. Y para obtener el paciente con apendicitis de menor prioridad, se invoca listaApendicitis.getMenorPrioridad(), que avanza mediante un puntero auxiliar hasta el último nodo de la lista (aux->siguiente == NULL) y retorna su paciente.

Explicación del comportamiento del programa

El programa inicia en **main.cpp** instanciando un objeto de la clase Controlador y mostrando un menú interactivo dentro de un bucle do-while que se repite hasta introducir la opción S:

1. **Parte superior:** Antes de cada lectura de teclado, la cabecera del menú consulta al controlador y muestra en tiempo real la cantidad de pacientes en la pila, en las Salas A, B, C y D, en los quirófanos de apendicitis y hernias, y en el árbol.
2. **Fase de generación y admisión (Opciones A, B y C):** Al seleccionar la opción A, se crean 12 pacientes con DNI y patología aleatorios y se apilan (permitiendo hasta 4 llamadas acumuladas, máximo 48 pacientes). La opción B recorre la pila mostrando únicamente el DNI y la enfermedad de cada paciente, ya que aún no han entrado al preoperatorio. La opción C permite vaciar la pila y restablecer el contador de generación.
3. **Fase de preoperatorio en salas (Opciones D, E, F y G):** Al pulsar D, todos los pacientes se desapilan, reciben su ID de gravedad y su habitación de planta, y se distribuyen equitativamente entre las Salas A y B (apendicitis) y las Salas C y D (hernias). El contador de la pila pasa a 0 y los contadores de las colas se actualizan. Las opciones E y F muestran los pacientes de cada par de salas incluyendo ya su ID y habitación asignada, mientras que G vacía las cuatro colas.
4. **Fase de quirófano en listas ordenadas (Opciones H, I, J y K):** La opción H desencola a todos los pacientes de las cuatro salas y los inserta por orden de prioridad en la Lista Apendicitis o en la Lista Hernias, dejando las colas a 0. Las opciones I y J imprimen el listado ordenado de cada quirófano, y la opción K localiza y muestra en pantalla al paciente de apendicitis de menor prioridad junto al de hernias de mayor prioridad.
5. **Reinicio del sistema (Opción L):** Libera toda la memoria dinámica de la pila, las cuatro colas y las dos listas, y limpia los vectores de control de IDs y habitaciones para comenzar una nueva simulación desde cero.

Bibliografía

- Departamento de Ciencias de la Computación, Universidad de Alcalá. *Apuntes y Ejemplos de clase*
- *Taboada Sánchez, A. M. [Programación ATS]. Programación en C++ [Lista de reproducción de YouTube]. Recuperado de https://youtube.com/playlist?list=PLWtYZ2ejMVJlUu1rEHLC0i_oibctkl0Vh&si=Fy-jUwNxplDFIW9x*